Rigid Body Interpolation via Dual Quaternions for Flow Matching

1 Introduction

This document details the mathematical representations and interpolation strategies for rigid bodies, specifically focusing on the application of Dual Quaternions (DQ) in the inference stage of Flow Matching models. We adopt the convention of global translation vectors $\boldsymbol{x}$ and rotation quaternions $\boldsymbol{q}$.

2 Quaternions and Exponential SLERP

2.1 Representation

A unit quaternion $\boldsymbol{q} \in \mathbb{H}_1$ represents a rotation in 3D space. We use the **scalar-first** convention:

$$\boldsymbol{q} = (w, x, y, z) = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k} \tag{1}$$

The identity rotation is $\boldsymbol{q}_{id} = (1, 0, 0, 0)$. The inverse of a unit quaternion is its conjugate $\boldsymbol{q}^* = (w, -x, -y, -z)$.

2.2 Exponential Map and Logarithm

To perform interpolation correctly on the $SO(3)$ manifold, we utilize the exponential and logarithmic maps. For a quaternion $\boldsymbol{q} = (w, \boldsymbol{v})$, the logarithm map to the tangent space $\mathfrak{so}(3)$ (represented as a pure quaternion vector) is:

$$\log(\boldsymbol{q}) = \begin{cases} (0, \mathbf{0}) & \text{if } \theta = 0 \\ \left(0, \frac{\theta}{\sin(\theta)}\boldsymbol{v}\right) & \text{if } \theta \neq 0 \end{cases}, \quad \text{where } \theta = \arccos(w) \tag{2}$$

The exponential map from a pure quaternion $\boldsymbol{\omega} = (0, \boldsymbol{\theta})$ back to the unit sphere is:

$$\exp(\boldsymbol{\omega}) = \left(\cos(\|\boldsymbol{\theta}\|), \sin(\|\boldsymbol{\theta}\|)\frac{\boldsymbol{\theta}}{\|\boldsymbol{\theta}\|}\right) \tag{3}$$

2.3 SLERP in Exponential Format

Standard Spherical Linear Interpolation (SLERP) between $\boldsymbol{q}_0$ and $\boldsymbol{q}_1$ with $t \in [0, 1]$ is formally defined using the exponential map of the relative rotation. Following Sola (2017), the trajectory is:

$$\text{SLERP}(\boldsymbol{q}_0, \boldsymbol{q}_1, t) = \boldsymbol{q}_0 \otimes \exp\left(t \cdot \log(\boldsymbol{q}_0^* \otimes \boldsymbol{q}_1)\right) \tag{4}$$

This formulation ensures the interpolation follows the geodesic (shortest path) with constant angular velocity.

3 Dual Quaternions

3.1 Representation

A dual quaternion $\hat{\boldsymbol{q}}$ consists of a real part $\boldsymbol{q}_r$ and a dual part $\boldsymbol{q}_d$, separated by the dual unit ϵ (where $\epsilon^2 = 0$):

$$\hat{\boldsymbol{q}} = \boldsymbol{q}_r + \epsilon \boldsymbol{q}_d \tag{5}$$

For rigid body transformations $SE(3)$, we use *unit dual quaternions*, satisfying the conditions $\|\boldsymbol{q}_r\| = 1$ and $\boldsymbol{q}_r \cdot \boldsymbol{q}_d = 0$.

3.2 Conversion: $(\boldsymbol{x}, \boldsymbol{q}) \to \hat{\boldsymbol{q}}$

We define the conversion for a **Global Translation** $\boldsymbol{x}$ followed by rotation $\boldsymbol{q}$ (or rotation then translation in global frame). Given a translation vector $\boldsymbol{x}$ and rotation quaternion $\boldsymbol{q}$, the dual quaternion is:

$$\begin{aligned} \boldsymbol{q}_r &= \boldsymbol{q} \\ \boldsymbol{q}_d &= \frac{1}{2} \boldsymbol{x}_{quat} \otimes \boldsymbol{q}_r \end{aligned} \tag{6}$$

where $\boldsymbol{x}_{quat} = (0, x, y, z)$ is the pure quaternion formed from the translation vector. [1]

3.3 Inverse Conversion: $\hat{\boldsymbol{q}} \to (\boldsymbol{x}, \boldsymbol{q})$

To recover the translation $\boldsymbol{x}$ and rotation $\boldsymbol{q}$ from $\hat{\boldsymbol{q}}$:

$$\begin{aligned} \boldsymbol{q} &= \frac{\boldsymbol{q}_r}{\|\boldsymbol{q}_r\|} \quad \text{(Normalization for stability)} \\ \boldsymbol{x}_{quat} &= 2\boldsymbol{q}_d \otimes \boldsymbol{q}_r^* \end{aligned} \tag{7}$$

The vector part of $\boldsymbol{x}_{quat}$ yields the translation $\boldsymbol{x}$.

Batch shapes (common in practice). If $\boldsymbol{x}$ has shape $(B, R, 3)$ and $\boldsymbol{q}$ has shape $(B, R, 4)$, then $\hat{\boldsymbol{q}}$ can be stored as an 8D tensor of shape $(B, R, 8)$ via concatenation $[\boldsymbol{q}_r, \boldsymbol{q}_d]$.

3.4 Dual Quaternion Algebra and Computation

To implement algorithms such as ScLERP and to maintain constraints on the SE(3) manifold, we define the fundamental algebraic operations for dual quaternions. Let $\hat{\boldsymbol{q}} = \boldsymbol{q}_r + \epsilon \boldsymbol{q}_d$ and $\hat{\boldsymbol{p}} = \boldsymbol{p}_r + \epsilon \boldsymbol{p}_d$ be two dual quaternions.

Dot Product The dot product is defined as the standard Euclidean inner product in $\mathbb{R}^8$. This is primarily used to check for antipodal alignment (i.e., $\text{sgn}(\hat{\boldsymbol{q}} \cdot \hat{\boldsymbol{p}})$):

$$\hat{\boldsymbol{q}} \cdot \hat{\boldsymbol{p}} = \boldsymbol{q}_r \cdot \boldsymbol{p}_r + \boldsymbol{q}_d \cdot \boldsymbol{p}_d \tag{8}$$

Hamilton Product Multiplication is distributive and follows the property $\epsilon^2 = 0$. The product $\hat{\boldsymbol{q}} \otimes \hat{\boldsymbol{p}}$ is:

$$\hat{\boldsymbol{q}} \otimes \hat{\boldsymbol{p}} = (\boldsymbol{q}_r \otimes \boldsymbol{p}_r) + \epsilon(\boldsymbol{q}_r \otimes \boldsymbol{p}_d + \boldsymbol{q}_d \otimes \boldsymbol{p}_r) \tag{9}$$

where $\otimes$ denotes the standard quaternion Hamilton product.

[1] Note: Some literature (e.g., Kenwright) defines $\boldsymbol{q}_d = \frac{1}{2}\boldsymbol{q}_r \boldsymbol{x}_{quat}$. That convention corresponds to translation defined in the *local* frame of the rotation. For Flow Matching in global coordinates, the form $\frac{1}{2}\boldsymbol{x}_{quat}\boldsymbol{q}_r$ is required.

Norm The magnitude of a dual quaternion is technically a dual number. However, for the purpose of projecting values back onto the Unit Dual Quaternion manifold (renormalization), we are concerned with the Euclidean norm of the real part (rotation) and the orthogonality of the dual part. A general normalization is applied as:

$$\|\hat{\boldsymbol{q}}\| = \|\boldsymbol{q}_r\| \tag{10}$$

A valid unit dual quaternion must satisfy $\|\hat{\boldsymbol{q}}\| = 1$ and the orthogonality constraint $\boldsymbol{q}_r \cdot \boldsymbol{q}_d = 0$.

Logarithm and Exponential These operations map between the Lie group SE(3) (represented by unit dual quaternions) and its Lie algebra $\mathfrak{se}(3)$ (represented by pure dual quaternions, or "twists").

For a unit dual quaternion $\hat{\boldsymbol{q}}$, let $\phi = \arccos(w_r)$ be the half-angle of rotation, and $\hat{\boldsymbol{n}}$ be the dual screw axis. The logarithm is:

$$\log(\hat{\boldsymbol{q}}) = \frac{1}{2}\hat{\boldsymbol{\theta}} = \frac{1}{2}(\boldsymbol{\theta}_r + \epsilon\boldsymbol{\theta}_d) \tag{11}$$

where $\hat{\boldsymbol{\theta}}$ is the dual vector representing the screw parameters (axis, angle, and pitch). In practice, this can be computed using the quaternion logarithm $\log \boldsymbol{q}_r$ and simple vector algebra for the dual part.

The exponential map takes a pure dual quaternion $\hat{\boldsymbol{v}} = \boldsymbol{v}_r + \epsilon\boldsymbol{v}_d$ (a twist) and maps it to a unit dual quaternion:

$$\exp(\hat{\boldsymbol{v}}) = \left(\cos(\|\hat{\boldsymbol{v}}\|), \sin(\|\hat{\boldsymbol{v}}\|)\frac{\hat{\boldsymbol{v}}}{\|\hat{\boldsymbol{v}}\|}\right) \tag{12}$$

Note that trigonometric functions cos and sin here are defined over dual numbers (e.g., $\sin(a + \epsilon b) = \sin a + \epsilon b \cos a$).

Power Function (t) The power operation $\hat{\boldsymbol{q}}^t$, fundamental to ScLERP, allows for interpolation along the screw motion path. It is defined via the exponential and logarithmic maps:

$$\hat{\boldsymbol{q}}^t = \exp(t \cdot \log(\hat{\boldsymbol{q}})) \tag{13}$$

This scales the screw angle and translation distance linearly by the factor t.

4 Inference via Overshoot-Backtrack Dual Interpolation

Standard ReQFlow inference generates translations and rotations via independent (decoupled) Euler integration steps. While efficient, this overlooks the natural screw motion coupling inherent in rigid body dynamics. To incorporate this geometric prior during inference, we propose a **Overshoot-Backtrack** strategy.

This predictor-corrector scheme first uses the decoupled flow to probe the vector field ("Overshoot"), and then relaxes the trajectory onto a coupled manifold path using Dual Quaternion interpolation ("Backtrack").

4.1 The Overshoot Step (Predictor)

Let the current state at time t be $T_t = (\boldsymbol{x}_t, \boldsymbol{q}_t)$. We aim to advance to $t+\Delta t$. First, we compute a temporary *overshoot* state, T_{over}, by stepping forward with a step size of $2\Delta t$ using the standard ReQFlow decoupled solver.

Using the predicted translation velocity $\boldsymbol{v}_{\theta,t}$ and angular velocity $\boldsymbol{\omega}_{\theta,t}$ (Eq. 10), and applying the exponential step scheduler derived in Eq. (15):

$$\boldsymbol{x}_{\text{over}} = \boldsymbol{x}_t + \boldsymbol{v}_{\theta,t} \cdot (2\Delta t), \tag{14}$$

$$\boldsymbol{q}_{\text{over}} = \boldsymbol{q}_t \otimes \exp\left(\frac{1}{2}(2\Delta t) \cdot \gamma e^{-\gamma t}\boldsymbol{\omega}_{\theta,t}\right). \tag{15}$$

Here, $T_{\text{over}} = (\boldsymbol{x}_{\text{over}}, \boldsymbol{q}_{\text{over}})$ represents a linearized projection of the trajectory at $t + 2\Delta t$.

4.2 The Backtrack Step (Corrector)

To determine the final state at $t + \Delta t$, we interpolate backwards from T_{over} to T_t. By converting these states into Unit Dual Quaternions (UDQ), we can employ coupled interpolation methods that respect the manifold structure of SE(3).

1. Conversion to Dual Quaternions. We convert the current and overshoot states into dual quaternions $\hat{\boldsymbol{q}}_t$ and $\hat{\boldsymbol{q}}_{\text{over}}$. Adopting the global translation convention:

$$\hat{\boldsymbol{q}} = \boldsymbol{q}_r + \epsilon\left(\frac{1}{2}\boldsymbol{x}_{\text{quat}} \otimes \boldsymbol{q}_r\right), \tag{16}$$

where $\boldsymbol{x}_{\text{quat}} = (0, \boldsymbol{x})$.

2. Coupled Interpolation. We compute the next state $\hat{\boldsymbol{q}}_{t+\Delta t}$ by interpolating between $\hat{\boldsymbol{q}}_t$ and $\hat{\boldsymbol{q}}_{\text{over}}$ with a factor $\lambda \approx 0.5$.

$$\hat{\boldsymbol{q}}_{t+\Delta t} = \text{Interp}(\hat{\boldsymbol{q}}_t, \hat{\boldsymbol{q}}_{\text{over}}; \lambda). \tag{17}$$

We propose three variants for the Interp$(\cdot)$ function:

- **Variant A: Dual Linear Blending (DLB).** Efficient and numerically stable. It approximates the screw motion by linearly blending the dual components and renormalizing:

$$\hat{\boldsymbol{q}}_{\text{dlb}} = \frac{(1-\lambda)\hat{\boldsymbol{q}}_t + \lambda \cdot \text{sgn}(\hat{\boldsymbol{q}}_t \cdot \hat{\boldsymbol{q}}_{\text{over}})\hat{\boldsymbol{q}}_{\text{over}}}{\|(1-\lambda)\hat{\boldsymbol{q}}_t + \lambda \cdot \text{sgn}(\hat{\boldsymbol{q}}_t \cdot \hat{\boldsymbol{q}}_{\text{over}})\hat{\boldsymbol{q}}_{\text{over}}\|}. \tag{18}$$

- **Variant B: Screw Linear Interpolation (ScLERP).** The theoretically exact geodesic on SE(3). It ensures constant screw axis velocities, ideal for high-precision trajectory requirements:

$$\hat{\boldsymbol{q}}_{\text{sclerp}} = \hat{\boldsymbol{q}}_t \otimes \exp\left(\lambda \cdot \log(\hat{\boldsymbol{q}}_t^* \otimes \hat{\boldsymbol{q}}_{\text{over}})\right). \tag{19}$$

- **Variant C: KenLERP (Hybrid Control).** Allows for regulating the degree of coupling. It blends the result of a decoupled step (standard Euler) and a coupled step (DLB/ScLERP). This is particularly useful for *diversity injection*. By sampling a mixing parameter β, the trajectory can fluctuate between the decoupled training distribution and the physically coupled geometric prior.

3. State Update. Finally, $\hat{\boldsymbol{q}}_{t+\Delta t}$ is converted back to $(\boldsymbol{x}_{t+\Delta t}, \boldsymbol{q}_{t+\Delta t})$ to proceed to the next inference step.

Algorithm 1 Overshoot-Backtrack Inference with Generic Interpolation

Require: Model $\mathcal{M}_\theta$, noise T_0, steps L, accel. γ, strategy $\in \{\text{DLB}, \text{ScLERP}\}$.

Initialize $t = 0, \Delta t = \frac{1}{L}$.
for step $= 1$ to L **do**
  // 1. Decoupled Overshoot (Predictor)
  Predict $\boldsymbol{v}_{\theta,t}, \boldsymbol{\omega}_{\theta,t}$ from $\mathcal{M}_\theta(T_t, t)$.
  $\boldsymbol{x}_{\text{over}} \leftarrow \boldsymbol{x}_t + \boldsymbol{v}_{\theta,t} \cdot (2\Delta t)$
  $\boldsymbol{q}_{\text{over}} \leftarrow \boldsymbol{q}_t \otimes \exp\left(\Delta t \cdot \gamma e^{-\gamma t} \boldsymbol{\omega}_{\theta,t}\right)$
  // 2. Coupled Backtrack (Corrector)
  $\hat{\boldsymbol{q}}_t \leftarrow \text{ToDual}(\boldsymbol{x}_t, \boldsymbol{q}_t)$
  $\hat{\boldsymbol{q}}_{\text{over}} \leftarrow \text{ToDual}(\boldsymbol{x}_{\text{over}}, \boldsymbol{q}_{\text{over}})$
  Sample $\lambda \sim \mathcal{N}(0.5, \sigma^2)$ {Stochastic diversity}
  if strategy == DLB **then**
    $\hat{\boldsymbol{q}}_{\text{next}} \leftarrow \text{DLB}(\hat{\boldsymbol{q}}_t, \hat{\boldsymbol{q}}_{\text{over}}, \lambda)$
  else if strategy == ScLERP **then**
    $\hat{\boldsymbol{q}}_{\text{next}} \leftarrow \text{ScLERP}(\hat{\boldsymbol{q}}_t, \hat{\boldsymbol{q}}_{\text{over}}, \lambda)$
  end if
  $\boldsymbol{x}_{t+\Delta t}, \boldsymbol{q}_{t+\Delta t} \leftarrow \text{FromDual}(\hat{\boldsymbol{q}}_{\text{next}})$
  $t \leftarrow t + \Delta t$
end for
return Generated backbone frame T_1

5 PyTorch Pseudo-code

```python
import torch

def quat_mul(a, b):
    """ Hamilton product for scalar-first quaternions (w, x, y, z) """
    aw, ax, ay, az = a.unbind(-1)
    bw, bx, by, bz = b.unbind(-1)
    w = aw*bw - ax*bx - ay*by - az*bz
    x = aw*bx + ax*bw + ay*bz - az*by
    y = aw*by - ax*bz + ay*bw + az*bx
    z = aw*bz + ax*by - ay*bx + az*bw
    return torch.stack([w, x, y, z], dim=-1)

def quat_conj(q):
    w, x, y, z = q.unbind(-1)
    return torch.stack([w, -x, -y, -z], dim=-1)

def to_dual_quat(t, q):
    """
    Convert Global Translation t and Rotation q to Unit Dual Quaternion.
    q_d = 0.5 * t_quat * q_r
    """
    # Normalize rotation
    q_r = q / (q.norm(dim=-1, keepdim=True) + 1e-8)

    # Create pure translation quaternion (0, t)
    zeros = torch.zeros_like(t[..., :1])
    q_t = torch.cat([zeros, t], dim=-1)

    # Global translation definition
    q_d = 0.5 * quat_mul(q_t, q_r)

    return torch.cat([q_r, q_d], dim=-1)

```

```python
def from_dual_quat(dq):
    """ Recover t and q from Unit Dual Quaternion """
    q_r = dq[..., :4]
    q_d = dq[..., 4:]

    # Re-normalize rotation
    q_r = q_r / (q_r.norm(dim=-1, keepdim=True) + 1e-8)

    # t_quat = 2 * q_d * q_r_conj
    q_t = quat_mul(2.0 * q_d, quat_conj(q_r))

    return q_t[..., 1:], q_r

def dlb_interpolate(dq1, dq2, t):
    """ Dual Linear Blending (Approximate ScLERP) """
    # 1. Handle antipodal ambiguity
    dot = torch.sum(dq1 * dq2, dim=-1, keepdim=True)
    sign = torch.sign(dot)
    sign[sign == 0] = 1 # Handle zero case
    dq2 = dq2 * sign

    # 2. Linear Blend
    dq_blend = (1 - t) * dq1 + t * dq2

    # 3. Renormalize (Unit Dual Quaternion Constraint)
    # The magnitude is determined by the real part norm
    q_r_blend = dq_blend[..., :4]
    norm = torch.norm(q_r_blend, dim=-1, keepdim=True) + 1e-8

    return dq_blend / norm
```

Listing 1: Dual Quaternion Implementation